

Tema 6: Arbres de cerca i maps

6.1 El problema de la cerca

Fins ara hem vist dues estructures lineals principals per guardar dades:

Contenidor	Accés	Cerca	Inserció al mig
<code>vector<T></code> (desordenat)	$\Theta(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
<code>vector<T></code> (ordenat)	$\Theta(1)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
<code>list<T></code>	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\Theta(1)$
<code>map<K, V></code>	—	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
<code>set<T></code>	—	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Queda clar: si la cerca és l'operació predominant, necessitem una estructura especialitzada. La solució és un **Arbre Binari de Cerca (BST)**.

6.2 Arbre Binari de Cerca (BST)

Un **BST** (*Binary Search Tree*) és un `BinTree<T>` que compleix una regla estructural addicional en cada node:

Invariant BST: Tots els valors del **subarbre esquerre** són **estrictament menors** que el node, i tots els del **subarbre dret** són **estrictament majors**.

[Gràfic interactiu disponible a la versió web de l'apunt]

Gràcies a l'invariant, el recorregut **inordre** (*esquerre* $\rightarrow$ *arrel* $\rightarrow$ *dret*) visita tots els valors **en ordre ascendent** en $\Theta(n)$.

6.3 Cerca en un BST

La cerca és l'operació més directa i eficient. A cada pas **descartem la meitat de l'arbre** sense ni explorar-la:

```
bool bst_search(const BinTree<int>& a, int x) {
    if (a.empty()) return false;
    if (x == a.value()) return true;
    return bst_search(a.value() < x ? a.left() : a.right(), x);
}
```

Observa el poder de la propietat BST: si $x < a.value()$, **sabem amb certesa** que x no pot estar al subarbre dret. El descartem completament sense explorar-lo.

[Simulació interactiva disponible a la web]

Nota Principal

En un BST **equilibrat**, la cerca costa $\mathcal{O}(\log n)$. En el pitjor cas (arbre degenerat/llista), pot arribar a $\mathcal{O}(n)$. La clau és mantenir l'arbre balancejat.

Mínim i Màxim

El valor mínim és sempre el node més a l'esquerra (baixem tot el camí cap a l'esquerra fins trobar un fill buit). El màxim, el més a la dreta.

```
int bst_min(const BinTree<int>& a) {
    assert(!bst.empty());
    if (a.left().empty()) return a.value();
    return bst_min(a.left());
}

int bst_max(const BinTree<int>& a) {
    assert(!bst.empty());
    if (a.right().empty()) return a.value();
    return bst_max(a.right());
}
```

6.4 Inserció en un BST

Com que `BinTree<T>` és **immutable**, no podem modificar l'arbre existent. La inserció **reconstrueix el camí** des de l'arrel fins al punt d'inserció, reutilitzant els subarbres que no canvien:

```
BinTree<int> bst_insert(const BinTree<int>& a, int x) {
    if (a.empty())
        return BinTree<int>(x);           // Cas base: aquí va el nou node
    if (x == a.value()) return a;          // Ja existeix, no inserim duplicat
    if (x < a.value())
        return BinTree<int>(a.value(),
                            bst_insert(a.left(), x), // Reconstruïm branca esq
                            a.right());              // Reutilitzem branca dreta
    return BinTree<int>(a.value(),
                        a.left(),
                        bst_insert(a.right(), x));    // Reutilitzem branca esq
}
```

[Simulació interactiva disponible a la web]

Fixeu-vos que els subarbres que **no estem travessant** es reutilitzen directament sense copiar-los. Gràcies a la immutabilitat funcional de `BinTree`, el compilador s'encarrega d'optimitzar la compartició de memòria (estructura persistent).

6.5 El contenidor `map<K, V>`

La STL de C++ ofereix el contenidor `map<K, V>`, un diccionari implementat internament com un **BST equilibrat** (arbre Vermell-Negre). Associa **claus úniques** (K) a **valors** (V).

```
#include <map>
using namespace std;

map<string, int> m;
```

```
m["un"] = 1;
m["deu"] = 10;
```

Internament, cada element és un `pair<K, V>` amb els camps `first` (clau) i `second` (valor).

L'operador `[]`: la porta d'entrada

L'operador `[]` és la forma més natural d'usar un `map`. Si la clau **no existeix**, la crea automàticament amb el valor per defecte del tipus (0 per a `int`, "" per a `string`).

Usar `m["clau"]` en un `map` `const` és **error de compilació** perquè no pot crear elements. Utilitza `m.find("clau")` o `m.at("clau")` en contextos `const`.

`find`: la cerca segura

```
map<string, int> m = {"un", 1}, {"deu", 10};

auto it = m.find("deu");
if (it != m.end()) {
    cout << "Valor associat: " << it->second << endl; // 10
}
```

`find` retorna un **iterador** apuntant al parell {clau, valor} si el troba, o a `m.end()` si no hi és. Cost: $\mathcal{O}(\log n)$.

`insert`: afegir sense sobre escriure

```
m.insert({20, "twenty"});
m.insert({20, "minus twenty"}); // NO substitueix si ja existeix!
```

El tipus de retorn d'`insert` és `pair<iterator, bool>`: l'iterador a l'element i `true` si s'ha inserit (`false` si ja existia).

6.6 El `map` com a acumulador

Un dels usos estrella del `map` és acumular i comptar. L'operador `[]` fa tota la feina:

1. Si la clau **no existeix** → la crea amb valor 0, llavors fa `++`.
2. Si la clau **existeix** → recupera el valor actual i fa `++`.

Exemple 1: Freqüència de paraules

```
#include <map>
#include <iostream>
using namespace std;

int main() {
    map<string, int> word_count;
    string word;
    while (cin >> word) {
        word_count[word]++; // <-- màgia pura!
    }
    for (auto it = word_count.begin(); it != word_count.end(); ++it) {
        cout << it->first << ": " << it->second << endl;
    }
}
```

Exemple 2: Agrupa paraules per longitud

```
map<int, vector<string>> by_length;
string word;
while (cin >> word) {
    by_length[word.size()].push_back(word);
}
```

[Simulació interactiva disponible a la web]

6.7 Iterar sobre un map

Els iterators de map recorren els elements **en ordre ascendent per clau** (perquè internament és un BST ordenat). L'operador `->` accedeix als camps **first** i **second** del parell:

```
map<string, int> m;
// ... (omplir el map)

for (auto it = m.begin(); it != m.end(); ++it) {
    cout << "clau: " << it->first
         << ", valor: " << it->second << endl;
}
```

O amb el bucle `for-each` modern (C++11):

```
for (const auto& [clau, valor] : m) {
    cout << clau << " → " << valor << endl;
}
```

6.8 El contenidor set<T>

Un `set<T>` és un map on només existeix la clau, sense valor associat. S'utilitza per: - **Eliminar duplicats** d'una seqüència. - **Comptar el vocabulari** (paraules úniques). - **Comprovar pertinença** en $\mathcal{O}(\log n)$.

```
#include <set>
using namespace std;

set<string> vocabulari;
string paraula;
while (cin >> paraula) {
    vocabulari.insert(paraula);
}
cout << "Paraules úniques: " << vocabulari.size() << endl;
```

Nota Principal

Si necessites claus **repetides** (multiset o multimap), C++ ofereix `multiset<T>` i `multimap<K, V>`. En un map normal, dues insercions amb la mateixa clau **no** afegixen un segon element.

6.9 Quan usar cada contenidor

Situació	Contenidor recomanat
Cerca freqüent en dades ordenades	<code>map<K,V></code> o <code>set<T></code>
Sense ordre, cerca màxima velocitat	<code>unordered_map<K,V></code> ($\mathcal{O}(1)$ amortitzat)
Accés per índex numèric	<code>vector<T></code>
Inserció/esborrat freqüent al mig	<code>list<T></code>
Col·lecció d'elements únics	<code>set<T></code>
Acumulador clau→comptador/llista	<code>map<K, vector<T>></code>